

US-1.1 / DOCUMENTATION TYPES MATRIX

Баримтжуулалтын дөрвөн төрөл

Гурван бодит төсөл ба RAG chatbot-д хэрэглэх шийдэл

1 Төслийг сонгосон үндэслэл

Би 2026 оны зун дадлагын ажлын хүрээнд хийсэн Document RAG Chatbot төслөө сонгосон. Хэрэглэгч PDF эсвэл текст оруулж, баримтын талаар асуулт асуухад систем холбогдох хэсэгт тулгуурлан хариулж, ашигласан эхийг харуулдаг. Өөрийн төслийг сонгосноор баримтжуулалтын төрлүүдийг бодит код, хэрэглэгчийн үйлдэлтэй холбон судалж, надад ойлгомжтой боловч шинэ уншигчид дутуу тайлбарлагдсан зүйлсийг илрүүлэх боломжтой.

2 Гурван төслийн дөрвөн төрлийн матриц

Seminar 1-ийн Getting Started, Tutorial, Reference, API ангиллыг хэрэглэв. API нь Reference-тэй давхцаж болох тул энд ерөнхий лавлахыг Reference, тодорхой интерфэйсийн гэрээг API гэж ялгав. Lecture-ийн User, Developer, Operations, Process нь уншигчаар ангилсан өөр хэмжээс юм.

Төслүүд: [fastapi/fastapi](#), [numpy/numpy](#), [encode/django-rest-framework](#). Доорх мөр бүр хэрэглэгчийн асуулт ба documentation-ийн байрлалыг хоёр өгүүлбэрээр тайлбарлана; холбоосуудыг 2026-09-11-нд шалгав.

FastAPI

Төрөл	Асуулт ба байрлал
Getting Started	“Хамгийн жижиг API-г хэрхэн ажиллуулж, хариуг нь харах вэ?” гэсэн асуултад First Steps хариулна. Байрлал нь /tutorial/first-steps/ бөгөөд анхны код, ажиллуулах алхам, шалгах үр дүнг нэгтгэсэн.
Tutorial	“JSON өгөгдөл хүлээн авах endpoint хэрхэн бүтээх вэ?” гэсэн суралцах зорилгыг Request Body гүйцэтгэнэ. Байрлал нь /tutorial/body/ бөгөөд өгөгдлийн загвараас хүсэлт боловсруулах хүртэл тайлбарлана.
Reference	“FastAPI-ийн аль хэсгийн дэлгэрэнгүй тайлбарыг хаанаас олох вэ?” гэсэн асуултад Reference индекс хариулна. Байрлал нь /reference/ бөгөөд class, parameter, response зэрэг сэдвийг лавлах байдлаар жагсаана.
API	“FastAPI class үүсгэхэд ямар параметр өгч болох вэ?” гэсэн интерфэйсийн асуултад class-ийн лавлах хариулна. Байрлал нь /reference/fastapi/ бөгөөд constructor-ийн параметр ба API гишүүдийг тодорхойлно.

NumPy

Төрөл	Асуулт ба байрлал
Getting Started	“NumPy-г анх хэрэглэж, массив хэрхэн үүсгэх вэ?” гэсэн асуултад Absolute basics хариулна. Байрлал нь /doc/stable/user/absolute_beginners.html бөгөөд эхлэгчид зориулсан үндсэн ойлголт, жишээг агуулна.
Tutorial	“Массивын shape, axis болон үйлдлүүдийг хэрхэн ашиглаж сурах вэ?” гэсэн асуултад Quickstart-ийн дасгалчилсан жишээнүүд хариулна. Байрлал нь /doc/stable/user/quickstart.html бөгөөд нэр нь quickstart ч Python мэддэг уншигчид зориулсан суралцах зорилготой тул Tutorial гэж ангилав.
Reference	“NumPy-ийн өгөгдлийн төрөл, массив, функцийн лавлах хаана байна вэ?” гэсэн асуултад Reference индекс хариулна. Байрлал нь /doc/stable/reference/ бөгөөд сэдвээр ангилсан лавлахын орох цэг болно.
API	“numpy.array функцийн dtype, сорь зэрэг аргумент юу хийдэг вэ?” гэсэн асуултад тухайн функцийн API хуудас хариулна. Байрлал нь /doc/stable/reference/generated/numpy.array.html бөгөөд signature, параметр, буцаах утга, жишээг өгнө.

Django REST framework

Төрөл	Асуулт ба байрлал
Getting Started	“Анхны хэрэглэгч, бүлгийн API-г хэрхэн босгох вэ?” гэсэн асуултад Quickstart хариулна. Байрлал нь /tutorial/quickstart/ бөгөөд төслийн үүсгэлтээс API-г шалгах хүртэлх богино замыг өгнө.
Tutorial	“Өгөгдлийг serializer ашиглан хэрхэн хөрвүүлж сурах вэ?” гэсэн асуултад Serialization tutorial хариулна. Байрлал нь /tutorial/1-serialization/ бөгөөд snippet кейсийг дагуулан хөгжүүлнэ.
Reference	“HTTP хариуны нэрлэсэн код, ангиллууд юу вэ?” гэсэн асуултад Status codes лавлах хариулна. Байрлал нь /api-guide/status-codes/ бөгөөд кодуудыг утгаар нь хайж ашиглахад зориулагдана.
API	“Request объектын data, query_params зэрэг гишүүн юу буцаадаг вэ?” гэсэн асуултад Requests хуудас хариулна. Байрлал нь /api-guide/requests/ бөгөөд хүсэлтийн объектын интерфэйсийг тайлбарлана.

3 RAG chatbot-д хэрэглэх кейсүүд

Getting Started: Төслийг анх ажиллуулах хүнд орчноо бэлдэх, текст оруулах, эх сурвалжтай хариу авах дарааллыг тайлбарлана. Үүний тулд үндсэн README-ээс docs/LOCAL_SETUP.md руу холбоос өгөх хэрэгтэй.

Tutorial: “Номын сан 18:00 цагт хаана” гэсэн текст оруулаад “Хэдэн цагт хаах вэ?” гэж асууж, хариуг эх хэсэгтэй тулгах нэг бүрэн урсгал бичнэ.

Reference: Retrieval-ийн threshold, lambda, useMMR тохиргооны default утга 0.1, 0.5, true болохыг хүснэгтээр тайлбарлана. Эдгээр нь chat.routes.ts-ийн утга бөгөөд хариултын чанарын баталгаа биш.

API: POST/chat-ийн session, documentIds, message, буцаах reply, sources, алдааны нөхцөлийг одоо байгаа route-оос баримтжуулна.

Бодит олдвор: Root README байхгүй, web README нь Vite template. PROJECT_BRIEF нь Supabase гэж бичсэн ч LOCAL_SETUP, Compose тохиргоо local PostgreSQL ашигладаг. Энэ зөрүү нь документацийг кодтой хамт шинэчлэх шаардлагыг харуулна.

Эх сурвалж: Seminar 01, US-1.1; Chinchilla, бүлэг 2, х. 9–24; Lecture 01, слайд 4. Төслийн нотолгоо: apps/api/src/modules/chat/chat.routes.ts, docs/LOCAL_SETUP.md, docker-compose.yml.

US-1.2 / AUDIENCE PERSONA CARD

Зорилтот уншигч: Тэнгис

Document RAG Chatbot

Энэ хэсэгт өөрийгөө төслийг анх үзэж байгаа хөгжүүлэгчийн байр сууринд тавьж, ямар тайлбар хэрэгтэйг тодорхойлов.

1 Persona Card

Талбар	Тодорхойлолт
Нэр	Тэнгис
Жишээ байгууллага	Nomad Learning Lab (persona-д зориулсан нэр)
Техникийн үүрэг	Web болон API холбож ажиллах хөгжүүлэгч
Уншигчийн суурь	TypeScript, React мэддэг; төслийн RAG ажиллагаатай танилцах шаардлагатай
Өдөр тутмын хэрэгсэл	Fedora, VS Code, Git, pnpm, DevTools, Docker Compose, curl
Зорилго	Local орчноос баримт оруулж, эх сурвалжтай хариуг web дээр гаргах
Унших хэв маяг	README-ээс эхэлж, командыг дагаж, алдааны тайлбарыг хайна

Миний тавьсан зорилго: Нэг баримт оруулаад асуулт асуух бүх алхмыг давтаж, алдаа гарвал аль хэсгээс шалгахаа ойлгодог болох.

Дам хэрэглэгч (end-end user): Хөгжүүлэгч энэ баримтыг уншиж системийг ажиллуулна. Харин chatbot ашиглах хүнд хариулт нь оруулсан баримтын аль хэсгээс гарсныг харах хэрэгтэй. Түүнд embedding-ийн дотоод ажиллагааг тайлбарлахаас илүү эх сурвалжаа шалгах боломж өгөх нь чухал.

2 Ижил төрлийн төсөлд тулгарсан гурван асуудал

P1. Нэвтрэлтийн cookie хадгалагдахгүй байх. Better Auth хэрэглэгч local орчинд ажилласан app Vercel дээр refresh хийхэд нэвтрэлтээ алддаг тухай мэдээлсэн. Тэнгист зориулсан зааварт web/API хаяг, session шалгах алхмыг нэгтгэх хэрэгтэй. [Stack Overflow: 79757950](#).

P2. Vector-ийн хэмжээс зөрөх. SpringAI/pgvector хэрэглэгч 1596 хэмжээс хүлээж байхад 4096 ирсэн алдаа мэдээлсэн. Энэ нь өөр stack-ийн жишээ; chatbot-ийн хэмжээс гэж үзээгүй. Тэнгис model солихдоо embedder ба SQL schema-ийн нийцлийг шалгах лавлахтай байх хэрэгтэй. [Stack Overflow: 78800728](#).

P3. PDF-ээс текст хоосон гарах. pdf-parse хэрэглэгч local орчинд текст гардаг ч Azure pipeline дээр хоосон гардаг тухай мэдээлсэн. Chatbot-ийн pdf-extractor.ts мөн хоосон текстэд алдаа өгдөг. Текст ялгалтыг AI хариултаас салгаж оношлох, текст paste хийх өөр замыг тайлбарлах шаардлагатай. [Stack Overflow: 60435500](#).

Эдгээр асуултаас өөрийн төслийн зааварт юуг тайлбарлах хэрэгтэйг харьцуулж үзэв. Бүгд миний төсөлд гарсан алдаа биш; холбоосуудыг 2026-09-11-нд шалгасан.

3 Өөрийн дүгнэлт

PDF оруулсны дараа мэдээлэл олдохгүй байвал файл боловсруулагдсан эсэх, зөв баримт сонгосон эсэх, хайлт үр дүн олсон эсэхийг дарааллаар шалгах заавар хэрэгтэй гэж үзлээ.

Тиймээс эхлэх заавартаа нэг бүтэн жишээ оруулж, ready төлөв болон хариултын эх хэсгийг хаанаас харахыг тайлбарлана. Энэ дүгнэлтийг ярилцлагаас бус, төслийн ажиллагаа ба дээрх асуултуудыг харьцуулж гаргав.

Эх сурвалж: Seminar 01, US-1.2; Bhatti, *Docs for Developers*, х. 15–17; Chinchilla, х. 7. Төслийн суурь: AGENTS.md, docs/LOCAL_SETUP.md, apps/api/src/modules/documents/pdf-extractor.ts.

US-1.3 / BEFORE-AFTER AUDIT

README-ийн ойлгомжгүй өгүүлбэрийг засах

Document RAG Chatbot

Эх файл: apps/web/README.md. Root README байхгүй тул байгаа Vite template README-ийн гурван бодит өгүүлбэрийг сонгов. Before нь үгчилсэн эх, After нь төслийн зориулалтад нийцүүлэн солих санал; эх төсөлд өөрчлөлт оруулаагүй.

1 J1. Хэрэгслийн нэрээс өмнө үр дүнг хэлэх

Before: “This template provides a minimal setup to get React working in Vite with HMR and some ESLint rules.”

Асуудал: React, Vite, HMR, ESLint-ийг мэддэг гэж үзсэн боловч энэ төслөөр юу хийж болохыг тайлбарлаагүй байна.

After: “Энэ веб аппад PDF эсвэл текст оруулж, баримтын талаар асуулт асууна. Хариултын хамт ашигласан эх хэсгийг харж шалгана. Жишээлбэл, ‘Номын сан 18:00 цагт хаана’ гэсэн текст оруулаад ‘Хэдэн цагт хаах вэ?’ гэж асууна.”

Засварын утга: Хэрэглэгчийн хийх үйлдэл, авах үр дүнг эхэнд тавьж, тодорхой жишээ өгсөн. Энэ нь Bhatti-ийн “Most Important Information First” зарчмыг хэрэглэсэн хувилбар.

2 J2. Template-ийн тайлбарыг бодит эхлэх замаар солих

Before: “The React Compiler is not enabled on this template because of its impact on dev & build performances.”

Асуудал: Compiler, dev, build-ийн утгыг тайлбарлаагүй. Мөн template-ийн ерөнхий тайлбар нь миний төслийг ажиллуулахад хэрэгтэй мэдээлэл өгч чадахгүй байна.

After: “Төслийн үндсэн хавтас дахь docs/LOCAL_SETUP.md заавраар database, API, web-ийг асаана. API ажилласны дараа <http://localhost:4000/health> хаягаар үйлчилгээнд хүрч байгааг шалгана.”

Засварын утга: Үндсэн onboarding хэсгээс баталгаажуулах compiler тайлбарыг хасаж, ажиллуулах зааварт чиглүүлэв. Compiler-ийн шийдвэрийг шаардлагатай бол хөгжүүлэгчийн хэсэгт тусад нь баталгаажуулна.

3 J3. Typescript ба lint-ийн ялгааг тайлбарлах

Before: “If you are developing a production application, we recommend updating the configuration to enable type-aware lint rules.”

Асуудал: Production, configuration, type-aware lint rules нь тайлбаргүй тул яг юуг шалгах, аль командыг ажиллуулах нь ойлгомжгүй байна.

After: “Кодын өөрчлөлтийг шалгахдаа үндсэн хавтсаас `npm run typecheck` ажиллуулна. Энэ нь TypeScript-ийн төрлүүд нийцэж буйг шалгана; жишээлбэл тоо авах хэсэгт текст дамжуулсан алдааг илрүүлж болно. Төрөлд тулгуурласан нэмэлт ESLint дүрмийг хөгжүүлэгч тусад нь шийднэ.”

Засварын утга: Бодит команд, зорилго, жишээ өгч, typecheck ба нэмэлт lint нь ялгаатайг тодруулав.

4 Өөрийн дүгнэлт

Эхний хувилбарууд хэрэгслийн нэрийг жагсаасан байсан бол засварласан хувилбарууд хийх үйлдэл, шалгах үр дүнг тайлбарлаж байна. README-ийн эхэнд төслийн зориулалтыг бичээд, ажиллуулах болон код шалгах зааврыг дараа нь байрлуулах нь зөв гэж үзлээ. Энэ удаа өөрөө харьцуулж зассан бөгөөд инженер биш уншигчаар шалгуулаагүй.

Эх сурвалж: Seminar 01, US-1.3; Bhatti, *Docs for Developers*, х. 3, 58; төслийн apps/web/README.md, docs/LOCAL_SETUP.md, package.json.

US-1.4 / WRITER-IN-THE-MIDDLE

Баримтжуулалт бичихийн ач холбогдол

Өөрийн төслийн дүгнэлт ба Corg.ly кейс

1 200 үгийн reflection

Би зун хийсэн RAG chatbot төслөө баримтжуулахдаа дараа нь унших хүнд юу хэрэгтэйг түрүүлж бодох ёстой гэж ойлголоо. Өөрийн бичсэн кодыг мэддэг ч шинэ хөгжүүлэгч хаанаас эхлэхээ мэдэхгүй байж болно. Түүнд баримт оруулаад эх сурвалжтай хариу авах дарааллыг тайлбарлана. Харин chatbot ашиглах хүнд хариултаа баримтын эхтэй тулгах боломж хэрэгтэй. Ингэж хөгжүүлэгчийн заавар эцсийн хэрэглээнд нөлөөлнө.

Marketing талд би чатботын боломжийг PDF асуултын бодит жишээгээр тайлбарлаж, OCR дэмждэг мэт амлалт өгөхөөс сэргийлнэ. Product талд баримт боловсруулах төлөвийг хэрэглэгч ойлгох эсэхийг шалгах шалгуур бичнэ. Sales талд үзүүлэх демог нэг текст, нэг асуулт, нэг эх сурвалжтай болгож давтахад хялбар болгоно. Support талд хоосон текст, нэвтрэлт, мэдээлэл олдоогүй хариут ялгасан заавар гаргана. DevRel талд шинэ хөгжүүлэгчийн анхны холболтыг тайлбарласан tutorial бэлдэнэ. Engineering талд README, тохиргоо, API гэрээний зөрүүг өөрчлөлтийн review дотор илрүүлнэ. Machine Readers талд хүсэлт болон хариуны талбаруудыг бүтэцтэй schema хэлбэрээр баримтжуулах санал гаргана.

Writer-in-the-Middle байрлалын хүн бүх багийн хүсэлтийг ганцаараа биелүүлэх боломжгүй. Би нээлтэд хэрэгтэй урсгалыг түрүүлж сонгон, хэсэг бүрийн эзэн, шинэчлэх шалтгаан, шалгах аргыг тохирно. Хэрэглэгчийн хүсэлтийг техникийн шийдэлтэй андуурахгүй, хэмжээгүй хугацааг амжилт гэж тайлагнахгүй. Миний ажлын үр дүнг хуудасны тоогоор бус, шинэ хүн анхны зөв хариунд хүрч, эхийг шалгаж, алдааны дараагийн алхмыг олж чадаж байгаагаар үнэлнэ. Ингэснээр багийн мэдлэг хадгалагдана.

2 Энэ ажлаас ойлгосон зүйл

1. Мэдлэгийн таамаг: Bhatti, х. 3. README-ийн HMR, ESLint нэрс шинэ хүнд бүтээгдэхүүний хэрэглээг ойлгуулахгүй тул эхэнд үр дүнг бичнэ.

2. Цаашид хэрэглэх арга: Chinchilla, х. 5. Хэсэг бүрийн эзэн, шинэчлэх шалтгаан, шалгах аргыг тохирч, API өөрчлөгдөхөд жишээг нэг review-д шинэчилнэ.

3. Бодит зөрүү: PROJECT_BRIEF-ийн Supabase тайлбар local PostgreSQL тохиргоотой зөрдөг. Root README-ээс одоогийн setup руу нэг эхлэх зам өгөх шаардлагатай.

Эх сурвалж: Seminar 01, US-1.4; Chinchilla, *Technical Writing for Software Developers*, х. 2–7; Lecture 01, слайд 9.

UE-1: Corg.ly нээлтийн өмнөх кейс

Bhatti-ийн х. 1–2-т Charlotte API-ийн нээлтэд бэлдэж байгаа ч хэрэглэгчид нь ажиллуулах заавар дутаж байна. Энэ нөхцөлд үндэслэн дараах уншигчийн жишээ болон README-ийн засваруудыг бэлтгэв. API-ийн өгүүлбэрүүдийг дасгалд зориулж бичсэн.

Part A. Persona Card

Нэр / байгууллага	Тэнгис / PawBridge Labs (кейсийн жишээ байгууллага)
Үүрэг / хэрэгсэл	Junior integration developer / Python, VS Code, Git, curl, Postman
Зорилго	Нэг sample дуу илгээж, орчуулгыг хүзүүвчийн аппад харуулах
Хүндрэл	Холболтын эрх, үр дүн бэлэн болох хугацаа, давхар хүсэлтийн эрсдэл
End-end user	Хүзүүвч ашиглах нохойн эзэн; орчуулга бэлэн эсэхийг мэдэхийг хүснэ

User story: Integration хийх хөгжүүлэгчийн хувьд нэг sample дуу илгээж, үр дүнг хүлээн авах алхмыг давтан хийж үзээд аппдаа холбохыг хүснэ.

Part B. Гурван өгүүлбэрийн Before/After

B1. Before: “Initialize the tenant-scoped client before inference.” Tenant, client, inference нь тайлбаргүй. **After:** “Дуу орчуулахын өмнө байгууллагынхаа холболтыг тохируулна. Жишээлбэл, сургалтын эрхээр холбогдоод нэг sample илгээнэ.”

B2. Before: “Poll the asynchronous job until terminal state.” Poll, asynchronous, terminal state нь ойлгомжгүй. **After:** “Орчуулга бэлэн эсэхийг ажлын дугаараар шалгана. Бэлэн бол үр дүнг, амжилтгүй бол алдааны тайлбарыг харуулна.” Үйлдлийг эхэнд тавьсан нь Bhatti, х. 58-ын зарчим.

B3. Before: “Replay idempotently on transient upstream failures.” Давтан оролдох нөхцөл, давхардлыг тайлбарлаагүй. **After:** “Холболт тасарвал өмнөх хүсэлт биелсэн эсэхийг эхлээд шалгана. Давтан хүсэлтийг ялгах тэмдэг дэмждэг бол sample-001 зэрэг ижил тэмдгээр дахин илгээнэ.”

Эдгээр засварт нэр томъёог хийх үйлдлээр нь тайлбарлав. Бодит API-ийн заавар болгохдоо төлөв, эрх, давтан хүсэлтийн дүрмийг хөгжүүлэгчээс нь тодруулах хэрэгтэй.

Part C. Charlotte-ийн Writer-in-the-Middle reflection

Би Charlotte-ийн оронд нээлтээс өмнө нэг sample-ээс үр дүнд хүрэх замыг түрүүлж баримтжуулна. Engineering-ээс хүсэлт, алдааны дүрэм; Product-оос хэрэглэгчийн зорилго; Support-оос давтагдах асуултыг авч нэгтгэнэ. Marketing, Sales-ийн амлалтыг энэ урсгалтай нийцүүлнэ.

Бүх асуултад Charlotte өөрөө хариулдаг байвал заавар бичсэн ч баг түүнээс хамааралтай хэвээр байна. Тиймээс хэсэг бүрийг хэн шинэчлэхийг тохирч, өөр хөгжүүлэгчээр зааврыг дагуулж үзэх хэрэгтэй. Гол нь шинэ хүн API-г холбоод, хүзүүвч ашиглах хүнд орчуулгын үр дүнг ойлгомжтой харуулж чаддаг болох юм.

Эх сурвалж: Bhatti, *Docs for Developers*, х. 1–3, 15–16, 58; Chinchilla, х. 5, 7.